

Full-Stack RAG Platform: Interview & Preparation Guide

1. How to Pitch the Project (Elevator Pitch)

When asked to introduce your project, use this concise and impactful summary:

"I designed and built an Enterprise-Grade, Microservices-based Multi-Document QA platform. It utilizes a hybrid approach to Retrieval-Augmented Generation (RAG) by combining traditional Vector Search (via ChromaDB) with Graph RAG (via Neo4j) to capture semantic relationships that vectors often miss. The backend is built with FastAPI, using PostgreSQL for session-based conversational memory. The frontend is a modern dashboard built with React and Tailwind CSS v4. Additionally, I implemented automated pipeline evaluation using the Ragas framework to track faithfulness and context recall, and dockerized the entire stack for simple, reproducible production deployment."

2. Technical Challenges Face & Solved

Challenge 1: Heavy Latency in Neo4j Entity Extraction (Graph RAG)

Problem: Extracting entities and relationships from PDF chunks using an LLM is a slow, blocking process. Running this synchronously during PDF upload caused requests to time out and froze the user interface.

Solution: I moved the Graph RAG entity extraction out of the main request-response cycle. I implemented FastAPI Background Tasks. When a PDF is uploaded, the API immediately extracts, chunks, and indexes the vectors into ChromaDB and returns a successful response to the user. The Neo4j graph entity builder is then kicked off asynchronously in the background, keeping the UI responsive.

Challenge 2: Docker Container Networking vs. Browser Client Context

Problem: When running inside Docker Compose, services communicate internally using container hostnames (e.g., api:8000). However, the React frontend executes inside the user's browser, not inside the Docker network. Hardcoding container names caused API call failures on the client machine.

Solution: I separated internal container communication from client-side communication. I configured the FastAPI container to expose port 8000 to the host, and configured the React client to dynamically detect the environment hostname (localhost vs production host), enabling seamless local and production deployments.

Challenge 3: Ingestion Quality & OCR for Scanned PDFs

Problem: Standard text extraction packages (like PyPDF2) fail completely when processing scanned documents or images inside PDFs, resulting in empty indexes or garbage text chunks.

Solution: I built a robust ingestion pipeline using pdfplumber for digital PDFs and fell back to pytesseract (OCR) with system-level libraries (tesseract-ocr, poppler-utils) for scanned images. I then added these system dependencies directly into the backend Dockerfile so that the environment works out-of-the-box.

Full-Stack RAG Platform: Interview & Preparation Guide

3. Top 10 Interview Q&A for Selection

Q1: Why did you combine Vector Search (ChromaDB) with Graph RAG (Neo4j)?

Vector search is excellent at finding local semantic similarity. However, it struggles with multi-hop reasoning or understanding complex relationships across different parts of a document. By using Neo4j, we extract entities and relationships to build a knowledge graph. This allows the system to traverse connections and answer relational questions that vector databases fail to answer.

Q2: What is Hybrid Search and how does your backend implement it?

Hybrid search combines dense vector retrieval (semantic search) with sparse keyword retrieval (lexical search like BM25). ChromaDB does the dense vector search using embeddings to capture context, while BM25 finds exact keyword matches. The results from both are merged and reranked to provide the LLM with the highest-signal context, improving response accuracy.

Q3: Why did you choose FastAPI over Flask or Django?

FastAPI is asynchronous by default (async/await), which is crucial for handling slow operations like calling external LLM APIs or uploading large PDFs without blocking other users. It also has built-in data validation using Pydantic and automatically generates OpenAPI/Swagger docs.

Q4: How does the system handle Conversational Memory?

The chat history is persisted in PostgreSQL. When a user sends a query, we retrieve past messages associated with their session_id. We format this history alongside the retrieved document chunks and feed it to the Groq LLM, allowing the model to understand references and follow-up questions.

Q5: What is the benefit of using React + Vite instead of Streamlit?

Streamlit is great for rapid prototyping but does not scale. It reruns the entire Python script on every interaction, making it slow. React + Vite provides a true Single Page Application (SPA) experience. It renders components modularly, allows custom styling (via Tailwind CSS v4), and isolates UI updates from the backend server logic.

Full-Stack RAG Platform: Interview & Preparation Guide

Q6: How did you configure Tailwind CSS v4 in this project?

Tailwind CSS v4 simplifies configuration by removing the need for `tailwind.config.js`. We used the new `@tailwindcss/vite` plugin in Vite's config and imported Tailwind directly in our `index.css` using `@import 'tailwindcss';`. This makes the build times significantly faster and enables CSS-first configurations.

Q7: Explain your Docker Compose setup and how the services communicate.

The system is composed of four dockerized services running on a shared bridge network (`rag_network`): `ui` (React served by Nginx on port 3000), `api` (FastAPI on port 8000), `neo4j` (Graph DB on port 7474), and `postgres` (Relational DB for memory on port 5432). The frontend in the browser communicates with the backend via host-exposed port 8000, while backend services communicate internally using container hostnames.

Q8: How do you detect and handle Hallucinations in generated responses?

In our `generate_answer` logic, we perform a verification step. We evaluate the generated answer against the retrieved chunks (ground truth context) to verify if the assertions made in the response exist in the source material. If there is a mismatch, we trigger the `hallucination_flag: true`, which our React frontend displays as a warning badge to the user.

Q9: What is the Ragas framework and what metrics do you track?

Ragas is a framework for evaluating RAG pipelines. We track Faithfulness (is the answer grounded strictly in the retrieved context?), Answer Relevance (does the answer directly address the user's question?), and Context Recall (did the retrieval system successfully fetch all required context?).

Q10: What would you improve or scale next in this project?

I would implement a hybrid query routing mechanism where simpler queries bypass the Neo4j graph lookup entirely to reduce latency. I would also add a caching layer (like Redis) for frequently asked questions to optimize API cost and improve response times.

4. Preparation Strategy to Get Selected

1. Be ready to share your screen and demo the live application. Pre-load some PDFs to show immediate results.
2. Emphasize the architectural separation of concerns (Frontend, API, Graph DB, SQL DB) and the use of Docker.
3. Highlight that you evaluated your pipeline using Ragas, which demonstrates a professional, metrics-driven approach to AI development rather than just building a simple toy app.